

Experiment No-09

Aim: Write case study of RTOS.

FreeRTOS

One of the most commonly used real-time operating systems in the embedded world is FreeRTOS.

Background

FreeRTOS (Free Real-Time Operating System) was created by Richard Barry in 2003 and currently is maintained by Amazon Web Services under an MIT licence. It is regularly voted as the most popular RTOS in embedded system surveys, and has been used on more than 40 processor architectures and in billions of devices, such as pacemakers and industrial PLCs, IoT sensors and automotive ECUs.

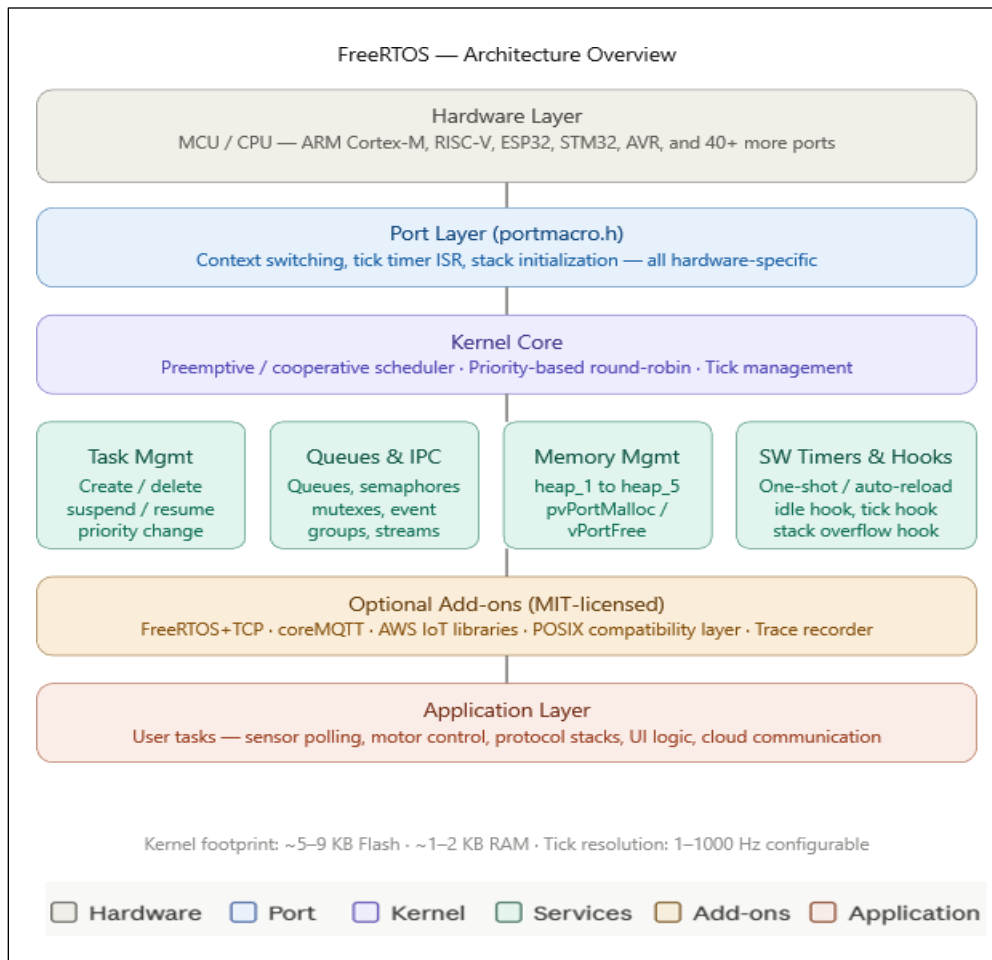
Core Design Philosophy

The single driving concept behind FreeRTOS is to do the bare minimum of a real-time operating system and do it right. The kernel itself is contained in only three C files: tasks.c, queue.c and timers.c making auditing and certification manageable. It excludes the file systems, networking, and device drivers in the kernel boundary.

- **Scheduling model:** FreeRTOS is based on a fixed-priority preemptive scheduler, which has optional time-slicing of tasks of the same priority. The priority ready task will always execute. When a task with a higher priority is ready (e.g., via an ISR posting to a queue) it preempts the task which is currently executing, at the next tick or at the end of the ISR.
- **Determinism:** Kernel operations have limited, predictable run time, that is essential to hard real-time guarantees in safety-critical systems.

Key Subsystems

- Task management allows creating tasks with individual stacks and priorities. The functions are all C functions that operate in an endless loop. The scheduler has a ready list of each priority, a delayed task list, and a suspended list.
- Inter-task communication (IPC) is built on queues. All of the semaphore, mutex, and event group are built over the queue mechanism. A mutex has priority inheritance to avoid priority inversion, a RTOS classic hazard where a low priority task holding a resource blocks a high priority task infinitely.
- Memory management provides five implementations of heaps (heap_1 to heap_5) with varying trade-offs, ranging between a trivial non-freeing allocator that can be used in systems with deep constraints, and a first-fit coalescing allocator that allows the use of non-contiguous memory regions.
- Software timers are executed by a special timer service task and may be one-shot or auto-reload, with the callbacks running in the context of that task.



Real-World Deployment Examples

Domain	Product	Use of FreeRTOS
Automotive	Bosch sensor ECUs	Task isolation between safety-critical sensor reads and CAN bus communication
Medical	Implantable cardiac monitors	Deterministic scheduling for alarm detection, certified by IEC 62304
Industrial IoT	AWS IoT Greengrass edge nodes	Paired with coreMQTT to cloud telemetry using sensors in factories.
Consumer	Amazon Echo Dot (some variants)	Manages wake-word detection pipeline alongside audio DSP tasks
Space	Various CubeSat platforms	Selected due to small footprint and radiation-tolerant MCU support.

Configuration and Footprint

Every behavioral aspect is controlled via FreeRTOSConfig.h, and anything unused is compiled out. A minimal configuration (no timers, no mutexes, heap_1) can fit in under 5 KB of Flash and under 1 KB of RAM, making it viable on an ATmega328 or STM32F0.

Certification and Safety

Amazon has FreeRTOS LTS (Long-Term Support) versions. A commercial variant SAFERTOS, by WITTENSTEIN high integrity systems, is a pre-certified version of FreeRTOS that has been proven to be IEC 61508 SIL 3 and ISO 26262 ASIL D, and is used where regulatory certification is required.

Strengths and Limitations

- **Strengths:** tiny footprint, MIT license, massive community, extensive port library, AWS backing, configurable tick rate, priority inheritance, well-documented API.
- **Limitations:** no in-built file system or networking (except by using add-ons), no memory protection between tasks except where the MCU has an MPU, and the MPU port variant is taken, no official symmetric multiprocessing (SMP support is provided in experimental branches), and debugging stack overflows requires either hook functions or a tool such as Tracealyzer.

Takeaway

FreeRTOS is successful due to the fact that it has exactly one problem which it addresses very well. It deals with scheduling of tasks with deterministic timing on resource constrained hardware and leaves the rest of the implementation to the developer. Its design demonstrates that a lean kernel that is correct is more useful in embedded systems than a rich one.